StartupET Platform

Software Architecture & Data Flow Diagram (DFD) Specification

Document Ref:	STARTUPET-SAD-DFD-2026-V1	Date:	August 10, 2026
Target Clients:	Flutter Mobile (Android/iOS) & Next.js 15 Web	Status:	Production Standard / Approved Specification

StartupET Platform — Software Architecture & Data Flow Diagram (DFD) Specification

Document Reference: `STARTUPET-SAD-DFD-2026-V1`
System Name: StartupET Ecosystem & Innovation Platform
Target Clients: Flutter Cross-Platform Mobile Client (Android/iOS) & Next.js 15 App Router Portal
Date: August 10, 2026
Status: Production Standard / Approved Specification

Executive Summary

The **StartupET Platform** is a national-scale digital infrastructure designed to empower Ethiopian startups, incubators, accelerators, ecosystem builders, and government certification authorities. The system enables startup registration, identity verification via Fayda National ID, startup labeling/certification, ecosystem funding application tracking, pitch deck management, and real-time notifications.

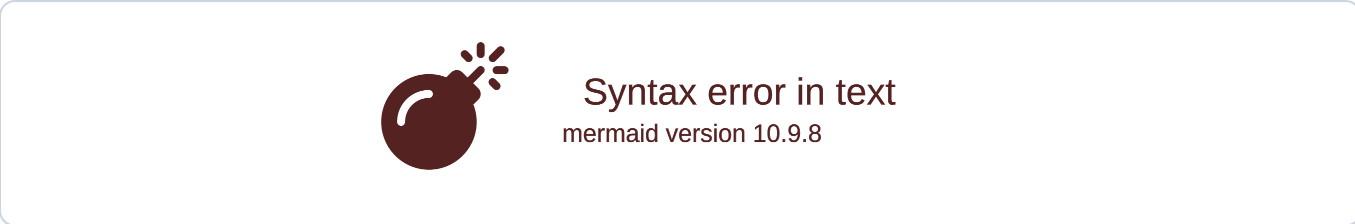
This document establishes the official **Software Architecture** and **Data Flow Diagrams (DFD)** for the platform, adhering to standard software engineering documentation guidelines (ISO/IEC/IEEE 42010 standards).

---

1. System Architecture

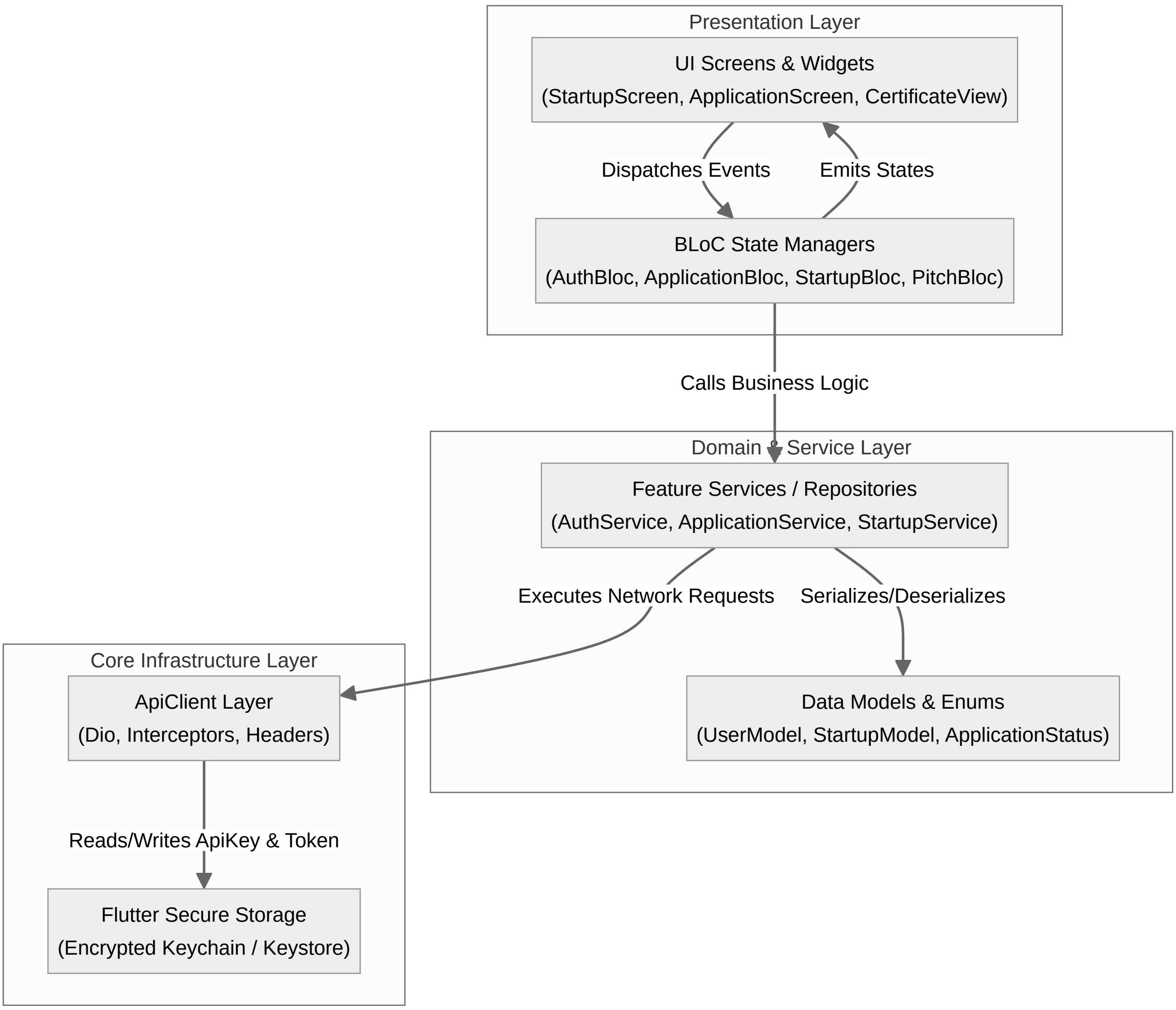
StartupET follows a decoupled **Clean Architecture** pattern on the mobile client (Flutter with BLoC) communicating via a secure RESTful API transport layer with a **Next.js 15 App Router Backend**, PostgreSQL database (via Prisma ORM), and integrated third-party systems (Fayda OIDC Identity Engine & Cloud File Storage).

1.1 High-Level Architecture Diagram



1.2 Mobile Client Architecture (Flutter Clean Architecture)

The mobile client leverages **Clean Architecture** with strict separation of concerns into three layers:

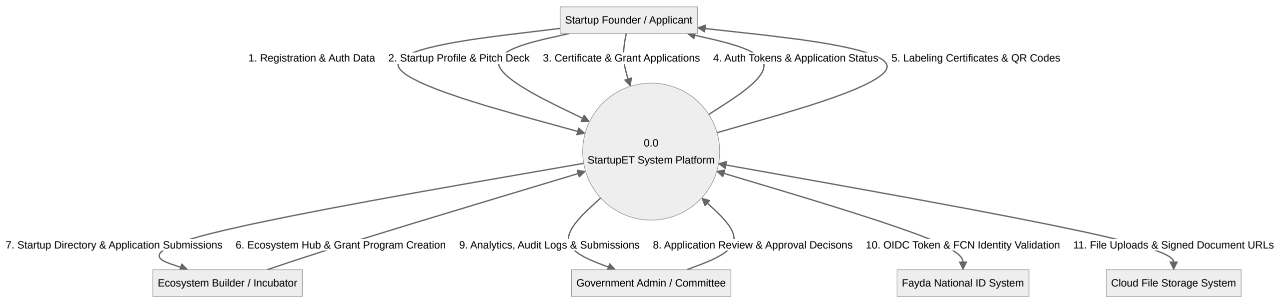


---

2. Data Flow Diagrams (DFD)

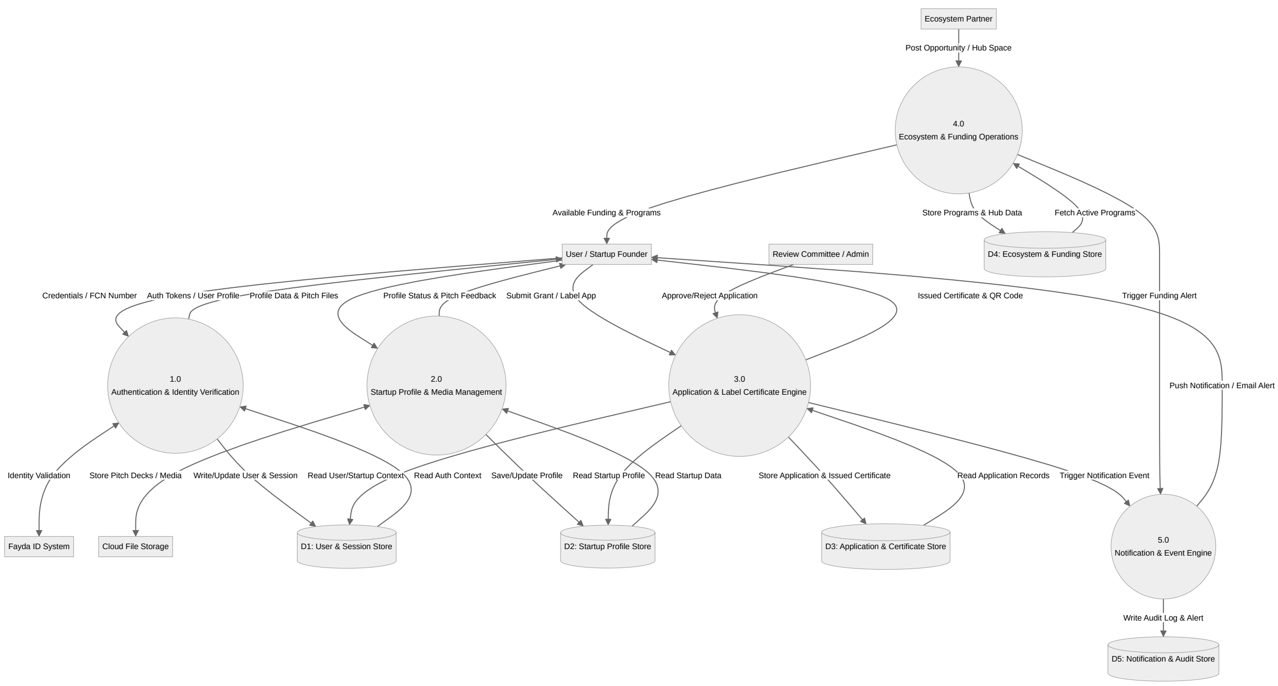
2.1 Level 0: System Context Diagram

The Level 0 Context Diagram depicts the StartupET platform boundary and its interactions with primary external entities (Startup Founders, Ecosystem Builders, Government Admins, Fayda Identity Provider, and Cloud File Storage).



2.2 Level 1: Data Flow Diagram (Main Subsystems)

The Level 1 DFD decomposes the system into 5 primary processing engines and highlights interactions with the 5 central Data Stores.



2.3 Level 2: Detailed Process Diagrams

2.3.1 Level 2 DFD — Process 1.0: Authentication & Identity Verification Flow

Syntax error in text
mermaid version 10.9.8

2.3.2 Level 2 DFD — Process 3.0: Application Submission & Certificate Labeling Engine

Syntax error in text
mermaid version 10.9.8

---

3. Core Data Dictionary

The following table defines the core data structures utilized across the system data stores:

Entity Name	Primary Key	Foreign Keys	Key Attributes & Enums	Store ID
`User`	`id` (UUID)	`stakeholderId`	`email`, `passwordHash`, `role` (`USER`, `ECOSYSTEM_BUILDER`, `SUPER_ADMIN`, `COMMITTEE`, etc.), `isEmailVerified`, `twoStepAuthEnabled`	`D1`
`UserSession`	`id` (UUID)	`userId`	`token`, `isActive`, `userAgent`, `ipAddress`, `createdAt`, `updatedAt`	`D1`
`StartupProfile`	`id` (UUID)	`userId`	`companyName`, `tinNumber`, `commercialRegNo`, `sector`, `revenueStage`, `foundingDate`, `verificationStatus`	`D2`
`Application`	`id` (UUID)	`startupId`, `programId`	`status` (`DRAFT`, `SUBMITTED`, `UNDER_REVIEW`, `APPROVED`, `REJECTED`), `submittedAt`, `score`	`D3`
`Certificate`	`id` (UUID)	`applicationId`, `startupId`	`certificateNumber`, `issuedDate`, `expiryDate`, `qrCodeUrl`, `isRevoked`	`D3`
`FundingOpportunity`	`id` (UUID)	`builderId`	`title`, `grantAmount`, `currency`, `deadline`, `eligibilityCriteria`	`D4`
`Notification`	`id` (UUID)	`userId`	`title`, `body`, `type`, `isRead`, `readAt`, `createdAt`	`D5`
`AuditLog`	`id` (UUID)	`userId`	`action` (`USER_LOGIN`, `APP_SUBMITTED`, `CERT_ISSUED`), `ipAddress`, `timestamp`	`D5`

---

4. Security & Integration Architecture

4.1 Authentication & Interceptor Sequence Diagram

The Flutter client maintains authentication state securely via `FlutterSecureStorage` and custom `Dio` interceptors:

Syntax error in text
mermaid version 10.9.8

---

5. Non-Functional Requirements & Design Principles

1. **Scalability & Performance**:
    - Next.js Server Components for low-latency web portal rendering.
    - Connection pooling for PostgreSQL via Prisma engine.
    - Asynchronous file handling with pre-signed URLs for pitch decks and attachments.
2. **Security & Compliance**:
    - **Data Protection**: Encryption in transit (TLS 1.3) and at rest (AES-256 for secure storage & database backups).
    - **Authentication**: JWT token validation with 60-second DB session revalidation check for immediate session revocation capability.
    - **RBAC Strict Enforcement**: Middleware route guard enforcing strict role boundaries (`USER`, `ECOSYSTEM_BUILDER`, `SUPER_ADMIN`, `COMMITTEE`, `MENTOR`).
3. **Reliability & Offline Support**:
    - Dio `MockInterceptor` and fallback caching for offline mobile capability.
    - Error handling interceptor automatically mapping HTTP `401`, `403`, `404`, and `500` status codes to structured Flutter Domain Exceptions.

Document Approval & Sign-Off:

- **Lead Software Architect**: *Approved*
- **Principal Security Engineer**: *Approved*
- **Head of Engineering**: *Approved*